

API_REFERENCE

HelixCode Notification API Reference

Overview

Complete API reference for the HelixCode notification system.

NotificationEngine

Creating a Notification Engine

```
engine := notification.NewNotificationEngine()
```

Methods

RegisterChannel

Registers a notification channel.

```
func (e *NotificationEngine) RegisterChannel(channel  
    NotificationChannel) error
```

Parameters: - channel: NotificationChannel - The channel to register

Returns: error

Example:

```
slackChannel := notification.NewSlackChannel(  
    "https://hooks.slack.com/...",  
    "#alerts",  
    "HelixBot",  
)  
err := engine.RegisterChannel(slackChannel)
```

SendNotification

Sends a notification using notification rules.

```
func (e *NotificationEngine) SendNotification(ctx
    context.Context, notification *Notification) error
```

Parameters: - ctx: context.Context - Context for cancellation - notification: *Notification - The notification to send

Returns: error

SendDirect

Sends a notification directly to specific channels (bypasses rules).

```
func (e *NotificationEngine) SendDirect(ctx context.Context,
    notification *Notification, channels []string) error
```

AddRule

Adds a notification rule.

```
func (e *NotificationEngine) AddRule(rule NotificationRule)
```

Notification Channels

Slack

```
channel := notification.NewSlackChannel(webhook, channel,
    username)
```

Parameters: - webhook: string - Slack webhook URL - channel: string - Slack channel (e.g., “#alerts”) - username: string - Bot username

Discord

```
channel := notification.NewDiscordChannel(webhook)
```

Parameters: - webhook: string - Discord webhook URL

Telegram

```
channel := notification.NewTelegramChannel(botToken, chatID)
```

Parameters: - botToken: string - Telegram bot token - chatID: string - Telegram chat ID

Email

```
channel := notification.NewEmailChannel(smtpServer, port,
    username, password, from)
```

Parameters: - smtpServer: string - SMTP server address - port: int - SMTP port - username: string - SMTP username - password: string - SMTP password - from: string - From email address

Generic Webhook

```
headers := map[string]string{
    "Authorization": "Bearer token",
}
channel := notification.NewWebhookChannel(url, headers)
```

Microsoft Teams

```
channel := notification.NewTeamsChannel(webhook)
```

PagerDuty

```
channel := notification.NewPagerDutyChannel(integrationKey)
```

Jira

```
channel := notification.NewJiraChannel(baseURL, email, apiToken,
    projectKey)
```

GitHub Issues

```
channel := notification.NewGitHubIssuesChannel(token, owner,
    repo)
```

Notification Types

```
const (
    NotificationTypeInfo      = "info"
    NotificationTypeSuccess  = "success"
    NotificationTypeWarning  = "warning"
    NotificationTypeError     = "error"
    NotificationTypeAlert    = "alert"
)
```

Notification Priorities

```
const (  
    NotificationPriorityLow      = "low"  
    NotificationPriorityMedium = "medium"  
    NotificationPriorityHigh    = "high"  
    NotificationPriorityUrgent  = "urgent"  
)
```

Reliability Features

Retry Mechanism

```
config := notification.DefaultRetryConfig()  
config.MaxRetries = 5  
config.InitialBackoff = 2 * time.Second  
  
retryableChannel := notification.NewRetryableChannel(channel,  
    config)
```

RetryConfig fields: - MaxRetries: int - Maximum retry attempts -
InitialBackoff: time.Duration - Initial backoff duration - MaxBackoff:
time.Duration - Maximum backoff duration - BackoffFactor: float64 -
Exponential backoff multiplier - Enabled: bool - Whether retries are enabled

Rate Limiting

```
limiter := notification.NewRateLimiter(10, 1*time.Second) // 10  
           per second  
rateLimitedChannel :=  
    notification.NewRateLimitedChannel(channel, limiter)
```

Predefined rate limiters:

```
limiter := notification.GetDefaultRateLimiter("slack")
```

Available defaults: - slack: 1/second - discord: 5/5 seconds - telegram: 30/second -
email: 10/minute - webhook: 100/minute

Notification Queue

```
queue := notification.NewNotificationQueue(engine, workers,  
    maxSize)  
queue.Start()
```

```
defer queue.Stop()
```

```
queue.Enqueue(notification, []string{"slack", "email"},  
    maxRetries)
```

Event System

Event Bus

```
bus := event.GetGlobalBus()
```

Publishing Events

```
bus.Publish(ctx, event.Event{  
    Type:      event.EventTaskFailed,  
    Severity:  event.SeverityError,  
    Source:    "task_manager",  
    TaskID:    "task-123",  
    Data: map[string]interface{}{  
        "error": "Connection timeout",  
    },  
})
```

Event Types

Task Events: - EventTaskCreated - EventTaskAssigned - EventTaskStarted -
EventTaskCompleted - EventTaskFailed - EventTaskPaused - EventTaskResumed -
EventTaskCancelled

Workflow Events: - EventWorkflowStarted - EventWorkflowCompleted -
EventWorkflowFailed - EventStepCompleted - EventStepFailed

Worker Events: - EventWorkerConnected - EventWorkerDisconnected -
EventWorkerHealthDegraded - EventWorkerHeartbeatMissed

System Events: - EventSystemStartup - EventSystemShutdown -
EventSystemError

Event Handler

```
handler := notification.NewEventNotificationHandler(engine)  
handler.RegisterWithEventBus(bus)
```

Metrics & Monitoring

```
metrics := notification.NewMetrics()

// Record operations
metrics.RecordSent("slack", duration)
metrics.RecordFailed("email")
metrics.RecordRetry("discord")

// Get metrics
current := metrics.GetMetrics()
fmt.Printf("Success rate: %.2f%%\n", metrics.GetSuccessRate())
fmt.Printf("Average response time: %v\n",
           current.AverageResponseTime)

// Channel-specific metrics
slackRate := metrics.GetChannelSuccessRate("slack")
```

Complete Example

```
package main

import (
    "context"
    "dev.helix.code/internal/event"
    "dev.helix.code/internal/notification"
)

func main() {
    // 1. Create event bus
    bus := event.GetGlobalBus()

    // 2. Create notification engine
    engine := notification.NewNotificationEngine()

    // 3. Register channels with retry & rate limiting
    slackChannel := notification.NewSlackChannel(
        "https://hooks.slack.com/...",
        "#alerts",
        "HelixBot",
    )

    // Add retry
```

```

retryConfig := notification.DefaultRetryConfig()
slackWithRetry :=
    notification.NewRetryableChannel(slackChannel,
        retryConfig)

// Add rate limiting
rateLimiter := notification.GetDefaultRateLimiter("slack")
slackFinal :=
    notification.NewRateLimitedChannel(slackWithRetry,
        rateLimiter)

engine.RegisterChannel(slackFinal)

// 4. Add notification rules
engine.AddRule(notification.NotificationRule{
    Name:      "Error Alerts",
    Condition: "type==error",
    Channels:  []string{"slack"},
    Priority:  notification.NotificationPriorityHigh,
    Enabled:   true,
})

// 5. Connect to event bus
eventHandler :=
    notification.NewEventNotificationHandler(engine)
eventHandler.RegisterWithEventBus(bus)

// 6. Publish events (notifications sent automatically)
bus.Publish(context.Background(), event.Event{
    Type:      event.EventTaskFailed,
    Severity:  event.SeverityError,
    TaskID:    "task-123",
    Data: map[string]interface{}{
        "error": "Database connection failed",
    },
})
}

```

Error Handling

All channel Send methods return an error:

```

err := channel.Send(ctx, notification)
if err != nil {

```

```
    log.Printf("Failed to send notification: %v", err)
}
```

Common errors: - “channel disabled” - Channel not configured - “rate limit” - Rate limit exceeded - “context canceled” - Context was canceled - HTTP status codes - External API errors

Best Practices

1. **Always use context** - Pass context for cancellation
 2. **Enable retries** - Wrap channels with retry logic
 3. **Use rate limiting** - Prevent API abuse
 4. **Monitor metrics** - Track success rates
 5. **Use event bus** - Automatic notifications
 6. **Handle errors** - Don't ignore errors
 7. **Test with mocks** - Use mock servers for testing
 8. **Use queues** - For high-volume scenarios
 9. **Set priorities** - Route urgent notifications properly
 10. **Document rules** - Clear notification rules
-

Testing

```
import "dev.helix.code/internal/notification/testutil"

func TestMyNotification(t *testing.T) {
    mockServer := testutil.NewMockSlackServer()
    defer mockServer.Close()

    channel := notification.NewSlackChannel(
        mockServer.URL,
        "#test",
        "bot",
    )

    notification := &notification.Notification{
        Title:    "Test",
        Message:  "Test message",
        Type:     notification.NotificationTypeInfo,
    }

    err := channel.Send(context.Background(), notification)
    assert.NoError(t, err)
}
```



```
requests := mockServer.GetRequests()  
assert.Equal(t, 1, len(requests))  
}
```

For more information, see: - [Testing Guide](#) - [Event Integration Guide](#) - [Configuration Reference](#)